Optimizing Forensic Data Availability and Retention of SDN Forensic Logs by Using Bloom Filter

Varun Sharma
International Institute of Information Technology
Hyderabad, India
varun.sharma@research.iiit.ac.in

Shatrunjay Rawat
International Institute of Information Technology
Hyderabad, India
shatrunjay.rawat@iiit.ac.in

***Abstract*—To perform network forensic analysis on an incident in software defined networking (SDN), logs are of the utmost importance. Without any logs, an investigator is not be able to complete and justify the forensic analysis. With the advancement of network and communication technologies, the volume of logs that needs to be collected and retained is growing exponentially big. The network administrators face the problems in maintaining such huge logs for longer period of time for forensic analysis. Network Providers rely on purging data based on a certain stipulated duration expected by the local rules for forensics evidence. SDN providers have limitations in managing such big data since the expense involved is commensurate with the duration of data retention. Here, in this work we propose a novel idea to reduce and summarize large forensic data sets for faster querying as well as reducing its space complexity by using bloom filters. Through this work we aim to propose a system which can deliver more optimized forensic data availability in SDN platform compared to existing systems.**

***Index Terms*—SDN, forensics, bloom filter, retention**

I. Introduction

Network Forensics is a stream of forensic science which deals with forensic activity in the area of computer networks which mainly involves identifying sources and paths of cyber attacks. This is the most important part of any cybercrime investigation and forensics. It primarily involves detecting all sources of attacks in the network, it can also be used as a proactive method to monitor and identify issues in the network infrastructure, overall performance, and bandwidth usage.

Security concerns in the network might start as something simple as traffic spikes or bottlenecks. These are, however, often ignored, especially in developing organizations, assumed to be caused by the application growth or increase in the number of users. But neglecting these issues can result in data breaches, loss of customer data, device crashing, etc.

With the exponential increase in network traffic every minute there has been proportional increase in the number of cyber attacks or malfunctions across the globe. To deal with such attacks and malfunctions, we need to capture and store the data for forensic data analysis.

We have a wide variety of network systems like WLAN, SAN, WAN, etc. of which SDN is the upcoming future of network system. SDN is gaining momentum in the networking industry due to its multiple advantages such as centralized, flexible, and programmable network management. The SDN framework consists of diverse devices like IoT devices, data centers and internet service providers spread across vast geographical areas. Each of these devices produce logs, which need to be analyzed and monitored to provide effective solutions for network security and comply with the regulations.

Current information systems are crowded with log files, created in multiple places (e.g., network servers, user monitoring applications, system services and utilities) for multiple purposes like maintenance, security issues, traffic analysis, legal data, and forensic analysis. Log files in such complex systems can grow quickly to humongous proportions causing difficulty to retain, monitor, control, and secure them for forensic analysis. Often they are required for prolonged periods of time.

Due to the overhead in managing archived data and expenses, data older than a certain retention period is purged in current systems. While analyzing the critical forensic incident to trace back and identify the potential source of attack, if the forensic data required is slightly older than the certain retention period policy and is purged already, we might completely miss out on gathering potential evidences for analyzing the source of the cyber attack path. This area is crucial based on the exigency of the threat forensic experts deal with, and attracts our attention to provide a solution to this problem.

The important factors need to be considered in terms of forensic data collection in SDN platform necessary for forensic analysis are system security, information security, confidentiality, and expense. System security deals with hardening of systems which should be tolerant of any security breach. Information security deals with ensuring the safety of collected network data while, confidentiality deals with maintenance of sensitive or private information in the collected network data. In these scenarios, enormous quantities of information need to be gathered for analytical or archive purposes. This requires the streamlining of the quantity of data to minimize data

transfer bandwidth utilization, maximize search throughput and reduce storage space requirements.

The expenses involved in network data storage is an important factor that should be considered for ensuring data availability in SDN platform for network forensics analysis. SDN providers cannot retain collected network data forever. Since the expense is commensurate with the duration of data retention, a system should be well designed by paying attention to the overall expenses involved.

Most of the local government policies on network log retention are based on the feasibility of the ISP/network providers' capacity to hold logs for certain period of time. This capacity is based on multiple challenges involved in the process like network log data management, data security, hardware requirements, expenses incurred. With our proposed system, duration to hold the forensic network logs can be enhanced to more than double the current capacity using almost similar hardware. This positively opens the scope for the local governments to consider this as an opportunity and they may revise such retention policies in crucial systems which currently varies from 1 month to 3 months in moderately busy network systems.

This paper is organized as follows:

'Section I' gives the introduction of data availability as crucial parameter for cyber crime evidence.

'Section II' gives the background of related works, which provide the basis of our research work.

'Section III' explains the proposed system.

'Section IV' demonstrates the experiment, results and analysis.

'Section V' concludes the research with open future research work scope followed by references.

II. Related Works

A. Security Approaches

In [1], authors emphasize on the evolution and continuous adaptability of SDN in the current network systems. Authors have discussed various advantages in SDN paradigm like ease of management, handling centralized connectivity across the nodes, cloud and network security concerns. This research work is aimed at providing the basics of SDN forensics and its advantages in cloud architecture. Authors also discussed various security approaches and categories in SDN platform and in the end, authors listed the security concerns in forensic based on SDN paradigm. This work pulls the attention towards the data security associated with data storage.

B. Effective Network Data Collection

In [3], authors discuss the scale at which the network is expanding and attracts the great number of intelligent attacks and malicious intrusions. Authors in this survey emphasized on the importance of network data collection mechanisms. It has been analyzed in this research work that an adaptive and effective data collection mechanism is still lacking. In this paper, authors listed down the current log collection mechanisms, architectures and performance of existing data collection systems. This work helped us focus more on data collection mechanisms and challenges associated with fast growing data.

As a fundamental procedure of network security measurement, network data collection executes real time network monitoring, supports network performance evaluation, assists network billing, and helps traffic testing and filtering. It plays a crucial and essential role while dealing with network intrusion detection and unwanted traffic control. But an adaptive and effective data collection mechanism that can be pervasively applied into heterogeneous networks is still lacking. They proposed number of assessment criteria, evaluated the performance of existing data collection mechanisms and summarized their characteristics. Furthermore, they figured out some open issues based on several investigations and did forecast the direction for future research.

C. Usage of Blockchain to Protect Data

In [4], authors propose an approach that leverages the features of a blockchain to make logs tamper proof. They introduce a potential solution to mitigate the challenges in digital forensics and cyber-security in SDN framework. Authors emphasize securing the forensic logs and implementing blockchain based approach to improve the security of log management in SDN platform and named this approach SDNLog-Foren. Authors further compare this system with other prevailing systems to prove its worth and advantages over others. This work throws light on cyber-security concerns while dealing with data. There are various attacks targeting a range of vulnerabilities on vital elements of this paradigm such as controller, Northbound and Southbound interfaces. In addition to solutions of security enhancement, it is important to build mechanisms for digital forensics in SDN which provide the ability to investigate and evaluate the security of the whole network system.

It provides features of identifying, collecting and analyzing log files and detailed information about network devices and their traffic. However, upon penetrating a machine or device, hackers can edit, even delete log files to remove the evidence about their presence and actions in the system. In this case, securing log files with fine-grained access control in proper storage without any modification plays a crucial role in digital forensics and cyber-security. This model is also evaluated with different experiments to prove that it can help organizations keep sensitive log data of their network system in a secure way regardless of being compromised at some different components of SDN.

D. Logs Availability for Analysis

In [5], authors have put the emphasis on how logs are important while dealing with run-time behavior of the software systems like debugging, system comprehensions and anomaly detection. To manage the unstructured nature and large size of logs, authors propose the solution with their system called LogAssist which assists administrators with log analysis. It significantly reduces the log size and improves the time

required to analyze the logs. This research work opens up the channel to think in this direction and design more optimized solutions.

It is a novel approach for assisting practitioners with log analysis, which aims to address all the three above-mentioned challenges. First, it parses the raw logs into abstracted log events (i.e., addressing the challenge related to unstructured logs). Then, it untangles the raw logs into meaningful event sequences (i.e., workflows) using certain grouping IDs commonly available in logs, to address the challenge related to intermixed event sequences. Finally, it leverages n-gram models to identify common event sequences and further uses the identified sequences to compress the logs into a much more concise representation (i.e., addressing the challenge related to the large size of logs).

III. Proposed System

Here we discuss our proposed system architecture in detail. Here we propose a system that leverages the strategy of integrating the data in bloom filters. Our proposed system offers a great advantage in retaining the forensic logs for a much longer duration.

Bloom filters are simple, space efficient data structures which work on the basis of a probabilistic algorithm to test membership in a huge set of hash functions. Bloom filter by design is a hashed data structure, hence we can rely on the system for reliability and safety of data. The system does not need any specific / customised firewall systems or security systems to secure confidential forensic data.

Bloom filters do not store the items themselves hence they use less storage space than the lower theoretical limit required to store the data correctly. Bloom filters exhibit an error rate which is inversely proportional to its size. The limited space available for bloom filter may cause high rate of false positives, but they do not have false negatives. The one-sidedness of this error turns out to be our benefit. When the Bloom filter reports the item as Found/Present, there is a small chance that it is not telling the truth, but when it reports the item as Not Found/Not Present, we know it's telling the truth. Most of the time, in the context where the query answer is expected to be Not Present, Bloom filters offer great accuracy along with space-saving benefits.

Alongside being storage efficient, time efficient and secured data structure, it is admissible IT evidence as per the local rules, since it complies with the requirement that the hashed data should produce same results as the raw data. To achieve this, we have optimized the data sets and size of bloom filters.

As we increase the number of hash functions, the false positive probability goes low as shown in the space efficiency of a bloom filter:

$$FP = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^{k} \approx (1 - e^{kn/m})^{k}) \qquad (1)$$

The requirement of designing 'k' different independent hash functions can be prohibitive for large 'k'. For a good hash function with a wide output, there should be a little correlation, if any, between different bit-fields of such a hash, so this type of hash can be used to generate multiple "different" hash functions by slicing its output into multiple bit fields. Alternatively, one can pass 'k' different initial values (such as 0, 1, ..., k-1) to a hash function that takes an initial value; or add (or append) these values to the key. For larger 'm' and/or 'k', independence among the hash functions can be relaxed with negligible increase in false positive rate.

The false positive probability 'p' is a function of number of elements 'n' in the filter and the filter size 'm'. An optimal number of hash functions

$$k = (m/n) \ln 2 \qquad (2)$$

System integrated in bloom filter enhances the availability of forensic logs with the similar hardware storage resources. It positively concludes that the introduced system can withhold comparatively more data as compared to the existing forensic data storage mechanisms. One method is to store the entire string to compare it with another string. This is a O(m+n) comparison. Some of very common data structures that adapt to the string data type pretty easily are Tries, Suffix trees, Binary Trees etc. These data structures have good complexity in theory, but it is not an easy task to optimise them for a given problem per se. The more efficient way is to store the hash of these strings. In practice Hashing has one of the best complexities with a fantastic O(1) constant search time. By analyzing the details as per the experimental values and disk size comparison done in 'Table I' & 'Fig. 9', we can conclude that the proposed system demonstrates satisfactory results.

TABLE I
Data Storage Analysis

Data Size(GB)	*Zip (GB)*	*Bz2 (GB)*	*Bloom Filter (MB)*
128	1.94	1.33	12.7
512	7.75	5.33	25
1024	15.5	10.67	52

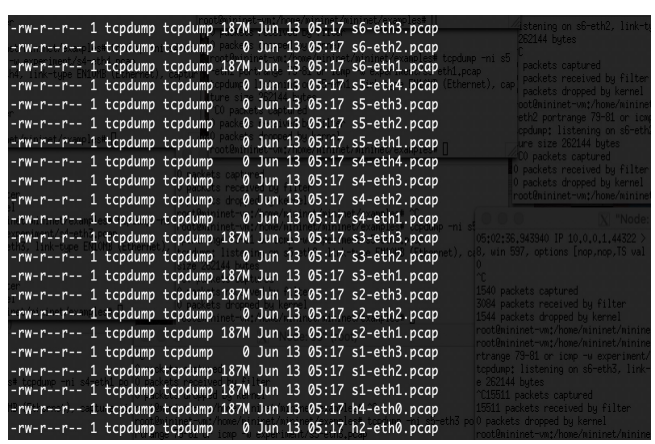

Fig. 1. Logs Collection

IV. Evaluation

In current systems, network administrators rely on purging the data older than a certain stipulated time expected by governmental guidelines and organizational rules. In addition, considering the retention period of logs for forensic analysis as constant factor, the rate at which the log volume is growing, it becomes a crucial factor in the system design for forensic analysis. In existing systems, the data storage capacities in network devices are very less, which is not ideal to hold historical data for more than a couple of months. With the reduced space and time complexity (refer 'Table VI'), inherited by design of bloom filter which states that a Bloom filter with 1% false positive rate requires only about 9.6 bits per element regardless of element size, the false positive rate can be reduced by a factor of ten each time 4.8 bits per element are added. By observing the less space utilization, its safe to claim that the expense involved in log collection and retention for network forensics logs are optimized by implementing bloom filter in current systems.

Generally, we choose convenient network packet capturing tools for data collection and further analysis. Wireshark and TCPdump are two classic applications of them. They rely on libpcap library and Berkeley Packet Filter (BPF).

The Proposed system is designed for SDN platform, where a controller is configured in such a way that each node/device in data plane keeps logging in a dedicated directory and keeps rolling these logs. In our controlled experiment, we specified that each file in the directory should not exceed 2 GB with a maximum limit and the directories contain the data of each device in pcap (from tcpdump) format.

In our experiment we use Mininet to simulate the SDN network with 1 controller, 6 switches and 4 hosts as shown in 'Fig. 2'. Controller is named as c0, switches are named as s1, s2, s3, s4, s5, s6 and hosts are named as h1, h2, h3, h4. We simulate a hybrid network topology by combining tree and mesh topology. All logs are centrally collected in a directory **experiment** with interfaces in their file names as shown in 'Fig. 1'. Different files are created with names, s1-eth1.pcap, s1-eth2.pcap, etc. which contain the packet captures of each interface of devices respectively. Simulated network dump and links are shown in 'Fig. 3' and 'Fig. 4' respectively.

The pox controller (SDN controller simulator used in our experiment) activation to handle hybrid network with mesh topology to avoid loops is shown in 'Fig. 5'. SDN Controller is simulated to run at port 6633.

Once we have the system set up and running as shown above, now we run a ping test from h1 to h4 to validate functional system simulation. Upon the successful ping test we can see the path traversed by the icmp packets:

$s3 \rightarrow s1 \rightarrow s2 \rightarrow s6$

Complete ping test with the tcpdump captures enabled in all interfaces terminals can be seen in 'Fig. 6'. Packets detailed path with interfaces are recorded as:

h1-eth0 $\rightarrow$ **s3-eth3** $\rightarrow$ **s3-eth1** $\rightarrow$ **s1-eth1** $\rightarrow$ **s1-eth2** $\rightarrow$ **s2-eth1** $\rightarrow$ **s2-eth3** $\rightarrow$ **s6-eth1** $\rightarrow$ **s6-eth3** $\rightarrow$ **h4-eth0**

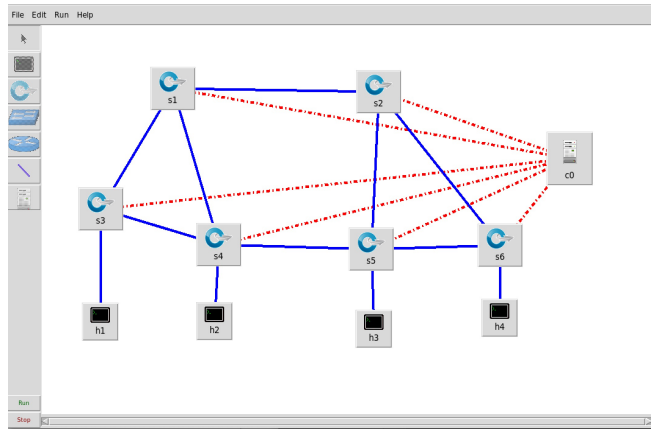

Fig. 2. Simulation Experiment Topology

Fig. 3. Network Dump

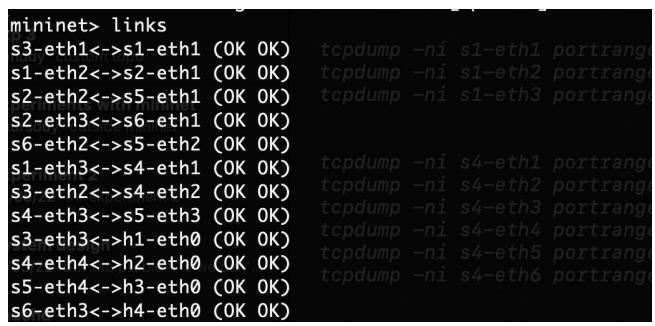

Fig. 4. Network Links

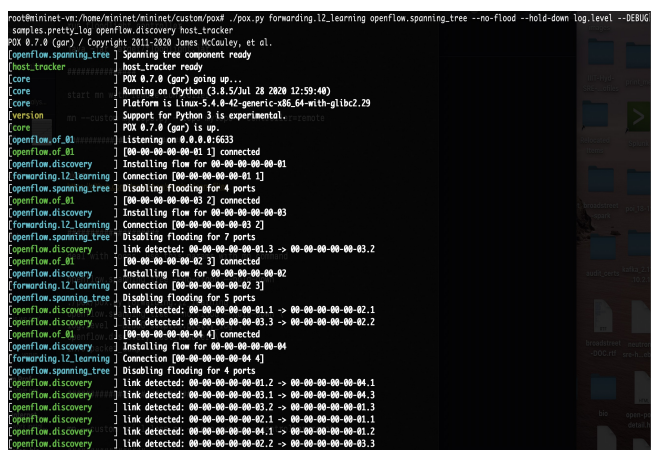

Fig. 5. SDN Controller Activation

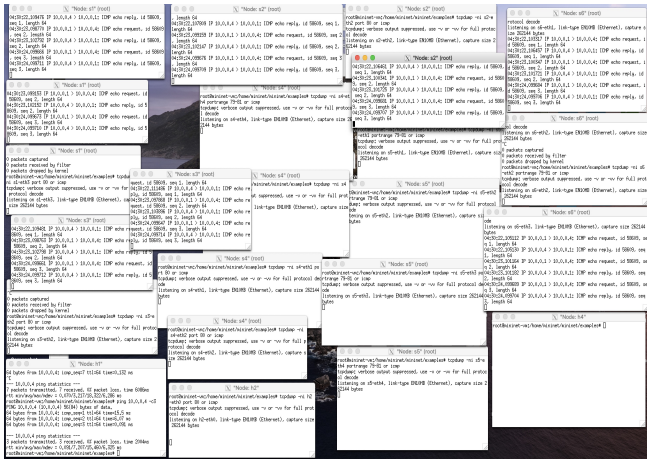

Fig. 6. Ping Test on Simulated Network

Upon successfully testing the network setup and smooth ping connectivity in all hosts, we extend the experiment with a web server serving on h4 and h1 being the client. We setup h4 as a python http server which is serving the traffic at port 80 and h1 as client downloading a FILE of 95 MB from h4(web server) using wget (wget http://10.0.0.4:80/FILE -o FILE). This experiment is run with active tcpdump captures at all interfaces of devices. After the FILE is downloaded at client h1, directory structure looks like as shown in 'Fig. 1'

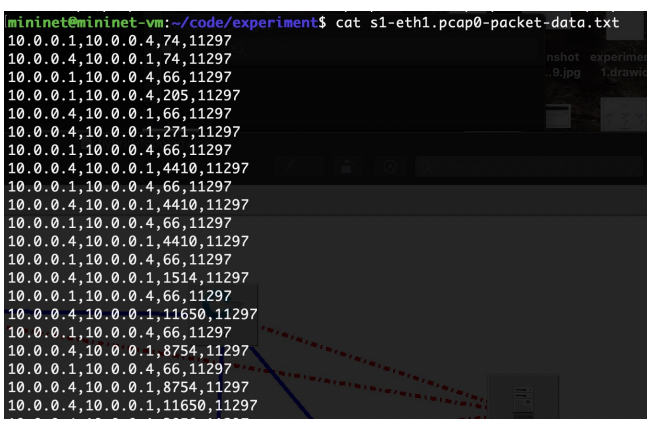

Fig. 7. Parsed Data

Now logs are arranged in a well defined and meaningful format. We parse each tcpdump pcap file and fetch important information of each packet like source of the packet, destination of the packet, size of the packet and total number of packets as shown in 'Fig. 7'. Next we focus on how to integrate this data in bloom filters, then design the query to find if the element exists in the BF or not and conclude the results. Now we integrate the parsed data as shown in 'Fig. 7' in bloom filter. We put the values fetched from each packet like packet size, packets count, source and destination IP of each packet into the bloom filters and then we query to check if that specific packet exists in any of the device pcap files. Please refer 'Table II', 'Table III', 'Table IV' and 'Table V' to analyse and create the Bloom filters that can avoid the saturation and deliver the optimised results for selecting the number of hash functions to be implemented in bloom filter for the respective false positive probability, in our experiments we are implementing bloom filters with 5 hash functions. Upon reaching the threshold to hit the bloom filter saturation, the system creates another .bin (bloom filter) file and so on in rolling fashion. From our experimental results we have observed and verified to accommodate significantly more forensic data i.e., retention of data for longer duration, using the same resources and maintenance expenses as compared to existing systems.

TABLE II
NUMBER OF HASH FUNCTIONS USED FOR VARIOUS FALSE POSITIVE PROBABILITIES

Suggested FPP of Bloom Filter (p)	***Hash Functions (k)***
3%	5
1%	7
0.1%*	10*
0.01%	13
0.001%	17

As per the IT Act rules, a hashed log is considered as valid network forensics evidence if the data represented in raw format also can have the same results/analysis. In our experiment to prove above acceptance, we have simulated a system which collects the SDN network logs and then we trace the path of a sample network packet using original raw logs and then the same logs were integrated in bloom filter which resulted the same network path upon search using same sample network packet.

Our proposed system can be of great importance for the Governments. We further throw some light on the crucial parameters of the proposed system and list the analysis and observations below in the experiments:

A. Logs Collection and structure

In our proposed system, data processing is done on the collected and structured data in a specified structured format. The structured data is used to generate the expected results. To achieve this, we apply the log rolling and parsing the meaningful information from the logs (tcpdump packet capture data in pcap files).

'Fig. 8' shows typical SDN system and forensic logs collection. Data Storage is arranged in a pattern with raw data which is used for real time monitoring, compressed logs which are retained for a stipulated time and then even older compressed logs are integrated in bloom filter for further retention for critical forensic analysis. With the acceleration of the 5G technologies and the promotion of the Internet of things (IoT) services, large scale and high-speed networks become the focus of current research and development. In order to collect and analyze network logs effectively, researchers and operators proposed many systems and applications as discussed in 'Section II'

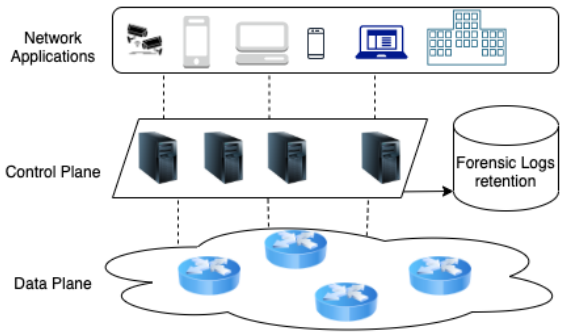

Fig. 8. SDN Architecture with Logs Flow

B. Rolling Logs

The logs are collected and arranged in a format where each directory represents the logs collected from respective devices. In our controlled experiment, we specified that each file in the directory should not exceed 2 GB with a maximum limit (general log rolling fashion).

C. IT & System Security

IT and system security are important aspects of securing the data from the vulnerabilities. Bloom filter by design is hashed data structure, hence we can rely on the system for reliability and safety of data. System does not need any customised firewall systems or security systems, to secure the confidential forensic data.

D. Expense

We have considered the expense as an important factor for the basis and motivation of our research. With our experiments data we have observed and verified to accommodate atleast 10 times more forensic data i.e. retention of data for longer duration, using the same resources and maintenance expenses compared to existing systems.

E. Admissible IT Evidence

Bloom filters do not store the items themselves and they use less space than the lower theoretical limit required to store the data correctly, and therefore, they exhibit an error rate. They have false positives but they do not have false negatives, and the one-sidedness of this error turned to our benefit. When the Bloom filter reports the item as Found/Present, there is a small chance that it is not telling the truth, but when it reports the item as Not Found/Not Present, we know it's telling the truth. So, in the context where the query answer is expected to be Not Present, Bloom filters offer great accuracy most of the time plus space-saving benefits. Apart from that it is admissible IT evidence as per the local rules, since it complies with the requirement of the hashed data should produce same results as the raw data, to achieve this we have optimised the data sets and size of bloom filters

F. Experiments for designing data structure of proposed system

'Table III' illustrates the behaviour of false positive probability with the number of hashes applied to the data. Rows with (*) mark is our favourable and optimised selected combination for the data structure i.e. 5 hashes.

TABLE III
PROBABILITY OF FALSE POSITIVES WITH HASH FUNCTIONS

Items in BF(n)	*hash fn(k)*	*bits in filter(m)*	*FPP (p)*
1 million	1	100 MB	1 in 801
1 million	3	100 MB	1 in 19 million
1 million*	5*	100 MB	1 in 106 billion*

'Table IV' illustrates the behaviour of FPP with the number of items in bloom filter. In this experiment we observed the data with 1 million items in bloom filter, 5 hashes and 100 MB bits in filter (size) is actually much more fault tolerant which is 1 false positive in 106 billion queries.

TABLE IV
PROBABILITY OF FALSE POSITIVES WITH NUMBER OF ITEMS IN BLOOMFILTER

Items in BF(n)	*hash fn(k)*	*bits in filter(m)*	*FPP (p)*
1 million*	5*	100 MB	1 in 106 billion*
2 million	5	100 MB	1 in 3 billion
3 million	5	100 MB	1 in 452 million
4 million	5	100 MB	1 in 108 million
10 million	5	100 MB	1 in 1 million

'Table V' illustrates the behaviour of FPP with the bits in filter (size). In the combined experiments we analysed the bit size of 2000MB with 1 million items and 5 hashes we can create an optimised data structure. We have marked the favourable combination with (*).

TABLE V
PROBABILITY OF FALSE POSITIVES WITH NUMBER OF BITS IN FILTER

Items in BF(n)	*hash fn (k)*	*bits in filter(m)*	*FPP (p)*
1 million	5	1000 MB	1 in 10 Quadri
1 million	5	2000 MB*	1 in 335 Quadri
1 million	5	3000 MB	1 in 2 Quinti
1 million	5	10000 MB	1 in 1 Sexti

'Table VI' illustrates the time complexity of the queries on the bloom filter organised data. We observed 10000 items in bloom filter produces optimised search results with 5 seconds for the query.

G. Analysis

We had run the experiments to analyze the proposed system with existing systems. Please refer the 'Table I' for the disk space utilization ratio for storing the logs with different compression and for the analysis, 'Fig. 9' shows the compression

TABLE VI
TIME COMPLEXITY

Number of Items in BF (n)	*Time Taken to search (ms)*
1000	1000
5000	2500
10000*	5100*
20000	11500
30000	17300

algorithms applied to 1024 GB of sample log directories of 50 MB files each. Important thing to note here is the disk size utilized by bloom filter does not depend on log file size, instead each element in bloom filter occupies around 5 Bytes of disk space for storing a file of 50 MB or 100 MB assuming the number of hash functions as 5 or 7. Our experiments are done on diverse directory sizes of 128 GB, 512 GB and 1024 GB with rolling file size of 50 MB / 100 MB / 1GB / 2 GB.

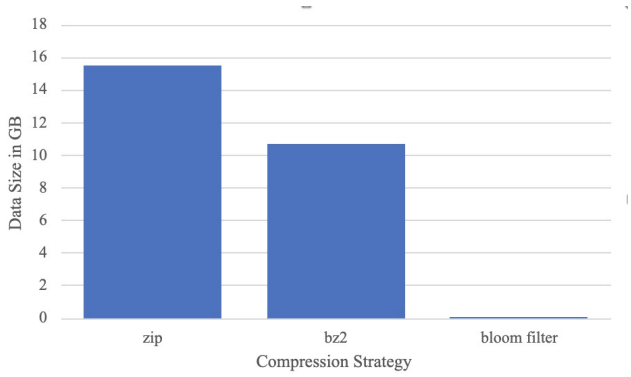

Fig. 9. Compression Analysis of 1024 GB sample data

As per the design, once the data is integrated in bloom filter it is irreversible and unrecoverable, though it has the potential to prove if the element definitely does not exist in the set of elements. We are leveraging the architecture of bloom filter to produce acceptable evidence with significantly less data storage requirements. This means it may not provide exact details of the logs in case of an incident but can provide potential evidence in determining the path of an attack, provided the forensic data older than retention period is organised as per our proposed design instead of purging it. Typically lets assume, if an SDN system generates logs at the rate of 100 GB per month and the disk available in the traditional system is 500 GB, we can not have any data beyond 5 months. Whereas using our system we claim that if we have 510 GB disk size available (10 GB additional) we can have 5 months raw data in the traditional formats(txt, pcap, zip, etc) and additional 5 months forensic hashed data. By suitably modifying the value of disk size and bloom filter we may alter the duration of forensic data retention as per the operational need.

V. CONCLUSION & FUTURE WORK

Post evaluation of our experimental results it is safe to conclude that the proposed system can withhold significantly more forensic data as compared to existing systems. We noticed that, if we use the proposed system, we need not purge all the data from the storage systems which is older than a certain stipulated retention period. Instead, we observed better forensic data availability, which might be considered as potential evidence in court of law to analyze the path traversed by the packet.

As a part of the future work we can plan to experiment on optimizing the bloom filter application by using enhanced multiple layered hashing, counting bloom filters & cuckoo filter. A More agile and robust system can be designed to handle the logs automatically and roll the files as a well organized scheme for different kinds of SDN traffic.

The proposed system can be enhanced and applied to other network systems which generate huge logs. This opens the scope of optimization of forensic data availability in different existing systems. We may further extend the experiments to demonstrate and compare the time complexity in different systems.

REFERENCES

[1] Q. Waseem, S. S. Alshamrani, K. Nisar, W. I. S. Wan Din, and A. S. Alghamdi, "Future Technology: Software-Defined Network (SDN) Forensic," *Symmetry*, vol. 13, no. 5, p. 767, Apr. 2021, doi: 10.3390/sym13050767. [Online]. Available: http://dx.doi.org/10.3390/sym13050767

[2] S. S. B. Renukuntla and S. Rawat, "Optimization of excerpt query process for Packet Attribution System," *2014 10th International Conference on Information Assurance and Security*, 2014, pp. 41-46, doi: 10.1109/ISIAS.2014.7064618.

[3] Donghao Zhou, Zheng Yan, Yulong Fu, Zhen Yao, "A survey on network data collection," *Journal of Network and Computer Applications*, Volume 116, 2018, Pages 9-23, ISSN 1084-8045, https://doi.org/10.1016/j.jnca.2018.05.004.

[4] P. T. Duy, H. Do Hoang, D. T. Thu Hien, N. Ba Khanh, V. Pham, "SDNLog-Foren: Ensuring the Integrity and Tamper Resistance of Log Files for SDN Forensics using Blockchain," *2019 6th NAFOSTED Conference on Information and Computer Science (NICS)*, 2019, pp. 416-421, doi: 10.1109/NICS48868.2019.9023852.

[5] S. Locke, H. Li, T. -H. P. Chen, W. Shang, W. Liu, "LogAssist: Assisting Log Analysis Through Log Summarization," *in IEEE Transactions on Software Engineering*, doi: 10.1109/TSE.2021.3083715.

[6] B. Siniarski, C. Olariu, P. Perry, T. Parsons, J. Murphy, "Real-time monitoring of SDN networks using non-invasive cloud-based logging platforms," *2016 IEEE 27th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC)*, 2016, pp. 1-6, doi: 10.1109/PIMRC.2016.7794973.

[7] R. Lu, L. Li, "Research on Forensic Model of Online Social Network," *2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, 2019, pp. 116-119, doi: 10.1109/ICCCBDA.2019.8725746.

[8] A. Asaduzzaman, A. Almohaimeed and K. K. Chidella,"Shared Entry Logger to Eliminate Duplicate Requests to SDN Controller," *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)*, 2019, pp. 0579-0584, doi: 10.1109/CCWC.2019.8666595.

[9] Wikipedia contributors. (2021, August 28). *Bloom filter. In Wikipedia, The Free Encyclopedia*. Retrieved 19:23, September 24, 2021, from https://en.wikipedia.org/w/index.php?title=Bloom_filter&oldid=1041007243